

Program Repair and Automated Debugging with Code LLMs

Abstract

Automated Program Repair (APR) and automated debugging represent critical frontiers in software engineering, promising to significantly reduce the immense manual effort historically required to maintain code reliability. For decades, the field evolved from heuristic and search-based techniques to template-driven and semantics-based methodologies. However, the emergence of Large Language Models (LLMs) pre-trained on vast, multi-lingual code corpora has precipitated a profound paradigm shift. This comprehensive survey systematically reviews the state-of-the-art in LLM-based APR and debugging. It categorizes the rapidly expanding body of literature into distinct methodological paradigms: fine-tuning and representation learning, prompt engineering and context augmentation, procedural and agentic frameworks, and neuro-symbolic reinforcement learning. Furthermore, this survey explores the integration of LLMs across the entire debugging lifecycle, including test-free fault localization, root cause analysis, and automated patch correctness assessment. Despite achieving unprecedented repair rates—often bridging the gap between isolated function-level fixes and complex, multi-file repository-level repairs—the field faces profound challenges. We critically examine the pressing issues of data contamination, benchmark leakage, and the "oracle problem" in patch validation, highlighting recent findings regarding the integrity of foundational benchmarks such as SWE-bench. Ultimately, this survey distills current empirical evidence to chart future research directions, emphasizing the necessity of verifiable reasoning, execution semantics alignment, and dynamic multi-agent collaboration in developing the next generation of autonomous software engineers.

Introduction

Software debugging and maintenance consume a disproportionate share of the software development lifecycle. Industry estimates suggest that developers spend nearly half of their programming time—approximately 48%—identifying, diagnosing, and rectifying software defects¹. Automated Program Repair (APR) aims to alleviate this extraordinary burden by automatically generating source code patches that resolve software bugs without introducing new regressions³.

Historically, traditional APR approaches were broadly classified into three primary categories: heuristic-based, constraint-based, and template-based methods⁵. Search-based techniques (such as GenProg) applied evolutionary mutations to code until a modified version successfully passed a provided test suite, while semantics-based methods extracted constraints via symbolic execution to synthesize mathematical or logical repairs³. Template-based tools (such as TBar) utilized predefined, human-crafted patterns mined from historical repositories to resolve common coding errors⁵. While these classical techniques demonstrated the

fundamental feasibility of automated repair, they suffered from severely restricted search spaces, poor scalability in enterprise environments, and an inherent inability to generalize beyond their predefined heuristics⁸.

To overcome these limitations, researchers pivoted toward learning-based approaches, treating APR as a Neural Machine Translation (NMT) task¹¹. By mapping buggy source code sequences (the source language) to fixed target sequences (the target language), deep learning (DL) models sought to automatically infer fix patterns. Systems such as SequenceR introduced copy mechanisms to mitigate the out-of-vocabulary problem common in source code, while CoCoNut advanced the state-of-the-art by utilizing context-aware architectures and Convolutional Neural Networks (CNNs) to represent buggy lines and their surrounding context separately⁷. Despite these architectural innovations, NMT-based models were intrinsically limited by the size and quality of parallel bug-fix datasets. They frequently generated syntactically invalid patches and struggled with complex logic errors because they learned superficial textual patterns rather than deep programmatic semantics¹⁰.

The advent of Large Language Models (LLMs)—built on the Transformer architecture and pre-trained on billions of tokens of open-source code and natural language—has revolutionized the software engineering landscape¹¹. Models such as OpenAI's GPT-4, Anthropic's Claude 3.5 Sonnet, and open-weight architectures like CodeLlama, DeepSeek-Coder, and StarCoder have demonstrated extraordinary zero-shot and few-shot capabilities in code generation, comprehension, and repair³. The immense parameter counts and broad pre-training of these models allow them to capture both the syntactic structure and the semantic intent of code across numerous programming languages¹⁷.

This paper provides a systematic, exhaustive survey of the deployment of LLMs in Automated Program Repair and automated debugging. We structure the research landscape into a unified taxonomy encompassing fine-tuning adaptations, prompt engineering with Retrieval-Augmented Generation (RAG), autonomous agentic pipelines, and neuro-symbolic reinforcement learning¹⁷. We extend beyond patch generation to examine the broader debugging workflow, analyzing LLM contributions to fault localization²² and patch validation²⁴. Finally, we critically assess the methodological threats facing the field, particularly the pervasive issue of data contamination in benchmarks like Defects4J and SWE-bench²⁶, providing a roadmap for rigorous, verifiably robust AI-driven software repair.

Methods

The integration of LLMs into the APR pipeline is not monolithic; rather, it spans a spectrum of methodologies characterized by varying degrees of parameter adaptation, context augmentation, and control authority over the repair loop¹⁷. We organize these methodologies into four primary paradigms for patch generation, followed by dedicated analyses of how LLMs are transforming fault localization and patch correctness assessment.

Fine-Tuning and Representation Learning

While pre-trained Code Language Models (CLMs) possess strong general coding capabilities, adapting them specifically to the distributions and structures of software bugs requires

specialized training. Fine-tuning CLMs on curated bug-fix datasets bridges the critical gap between general code generation and targeted bug rectification²⁰.

Full Parameter Fine-Tuning and Parameter-Efficient Fine-Tuning (PEFT): Early empirical evaluations demonstrated that off-the-shelf CLMs can significantly outperform traditional DL-based APR techniques. However, explicitly fine-tuning these models on APR datasets yields dramatic improvements. Comprehensive studies fine-tuning models like CodeT5, CodeGen, and PLBART revealed performance gains ranging from 31% to 1,267%, enabling them to fix up to 164% more bugs than state-of-the-art NMT baselines²⁰.

Because full-parameter fine-tuning of billion-parameter models is computationally prohibitive and prone to catastrophic forgetting, researchers have increasingly adopted Parameter-Efficient Fine-Tuning (PEFT) techniques, such as Low-Rank Adaptation (LoRA) and IA3. These methods freeze the base LLM weights and inject trainable rank decomposition matrices into the transformer layers, drastically reducing memory overhead¹⁸. Interestingly, some studies observe that full fine-tuning can sometimes decrease benchmarking performance due to differing data distributions and overfitting, making PEFT a more robust alternative for maintaining the model's generalized pre-trained knowledge while adapting it to APR³⁰.

Domain-Specific and Self-Supervised Fine-Tuning: A major limitation of classical supervised fine-tuning is its reliance on historical, cross-project bug fixes. This deprives the model of project-specific context, such as custom APIs, local variables, and domain-specific architectural patterns. The "plastic surgery hypothesis" posits that the ingredients needed to fix a bug typically already exist within the buggy project itself³¹. To exploit this, techniques like FitRepair employ domain-specific training strategies. By utilizing Knowledge-Intensified fine-tuning—aggressively masking tokens in the original buggy project—the LLM is forced to learn project-specific programming styles and identifiers, resulting in a state-of-the-art repair rate on the Defects4J benchmark³¹.

Taking a self-supervised approach, the SelfAPR framework generates massive synthetic training datasets by applying automated perturbations to working versions of the target software. This paradigm forces the model to learn project-specific idioms without relying on mined historical commits¹⁴. Crucially, SelfAPR executes these perturbed programs to extract compiler and test execution diagnostics, appending these dynamic signals to the training input. This effectively shifts the learning paradigm from static text-to-text translation to diagnostic-guided repair³³. Similarly, the RewardRepair architecture incorporates compilation and test execution states directly into the loss function during backpropagation. By penalizing the network for generating syntactically invalid or overfitted patches, RewardRepair increases the yield of compilable patches to 45.3%, significantly improving the efficiency of the validation phase¹⁵.

Meta-Learning for Low-Resource Bugs: Software bugs exhibit a highly imbalanced, long-tailed distribution. While syntax errors or off-by-one errors might be common, complex concurrency defects or domain-specific logic bugs are rare. Standard fine-tuning causes models to overfit on frequent error types and fail on low-resource bugs³⁶. Frameworks like Meta-APR address this by integrating the Reptile meta-learning algorithm with models like

CodeT5. Meta-APR first meta-trains on high-resource bug classes to learn generalized, error-specific initializations. This enables rapid and robust adaptation to target low-resource bug types using as few as 10 to 100 few-shot examples³⁶.

Prompt Engineering and Context Augmentation

For closed-source or extremely large LLMs (e.g., GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro), parameter adaptation is often infeasible due to API restrictions or computational costs. Instead, researchers rely on advanced prompt engineering and context augmentation to steer the model's zero-shot or few-shot reasoning capabilities.

Zero-Shot, Few-Shot, and Cloze-Style Prompting: Basic zero-shot prompting instructs the LLM to generate a fix given a buggy function. However, framing APR as a standard left-to-right code generation task discards the contextual information following the bug. The AlphaRepair framework pioneered cloze-style (infilling) APR by stripping out the buggy code and forcing the LLM to predict the missing tokens based on both the prefix and suffix context⁶. This approach requires no fine-tuning and directly leverages the bidirectional context modeling capabilities of pre-trained models.

To address complex logic errors, ThinkRepair enhances standard prompting by eliciting a Chain-of-Thought (CoT). It requires the LLM to output a step-by-step logical analysis of the bug before generating the fixed code. Furthermore, ThinkRepair constructs a pre-fixed knowledge pool, dynamically selecting high-quality reasoning examples for few-shot learning, which drastically improves the model's analytical depth¹⁰.

Retrieval-Augmented Generation (RAG): Because LLMs suffer from context window limitations and cannot memorize the entirety of a large enterprise codebase, Retrieval-Augmented Generation (RAG) dynamically fetches relevant context to ground the generation process³⁹. In APR, frameworks like RAP-Gen utilize a dense retriever to fetch semantically similar historical bug-fix pairs from an external codebase, augmenting the prompt with these exemplars to guide the patch generator⁴².

SelRepair extends this paradigm with a dual RAG module and a dynamic selection gate that evaluates the semantic and structural (AST-based) similarity of retrieved information. This gate acts as a rigorous filter, ensuring only highly relevant tokens are passed to the LLM, thereby reducing prompt length and accelerating inference time by up to 13.77% while preserving high accuracy⁴³. RAG is also pivotal in resolving multi-file dependencies and API misuses that require global repository awareness⁴⁵. In specialized domains, SimLLMRepair applies RAG to Model-Based Development (MBD), translating visual Simulink models into structured JSON and retrieving mechanical fault detection rules to guide the LLM²⁵.

Conversational and Feedback-Driven Repair: Standard Generate-and-Validate (G&V) pipelines sample an LLM independently multiple times, leading to repetitive, redundant patch generation³⁸. Conversational APR, exemplified by ChatRepair, replaces this isolated sampling with a stateful dialogue⁴⁷. If a generated patch fails the test suite, ChatRepair captures the compiler or runtime error message and appends it to the chat history. This feedback loop prevents the LLM from repeating identical mistakes and allows it to progressively refine its logic, achieving state-of-the-art fix rates on Defects4J at an average inference cost of just

\$0.42 per bug⁸.

DebugRepair takes conversational APR further by capturing intermediate runtime states rather than relying solely on binary pass/fail test outcomes. It semantically purifies failing tests to strip out noise, instruments the program to trace variable values during execution, and serializes these dynamic traces into the prompt⁴⁸. By contrasting observed runtime values against expected behaviors in the prompt, the LLM is forced to engage in evidence-based deductive reasoning rather than surface-level pattern matching⁴⁸.

Procedural and Agentic Frameworks

As bug complexity scales from isolated, single-function errors to multi-file, repository-level issues (such as those found in the SWE-bench dataset), static prompts and simple conversational loops become insufficient. The frontier of LLM-based APR is now dominated by autonomous software engineering agents that orchestrate planning, tool-use, and multi-step reasoning⁴⁹.

Autonomous Agent Architectures: RepairAgent was one of the first systems to conceptualize the LLM as a fully autonomous agent rather than a passive code translator⁵⁰. Operating via a dynamic Finite State Machine (FSM), RepairAgent freely interleaves information gathering, fault localization, and patch validation. It is equipped with a suite of 14 deterministic tools—enabling it to search the codebase for repair ingredients via grep, read specific file lines, run compilation tasks, execute tests, and apply patches. At each cycle, the agent observes the terminal output of its previous action, updates its dynamic prompt, and selects the next optimal tool⁵⁰.

Similarly, Passerine implements a ReAct (Reasoning and Acting) loop adapted specifically for enterprise-level environments, successfully navigating the idiosyncrasies of Google's internal issue trackers (GITS) and proprietary build systems⁵⁴. Agentless takes a slightly different approach by employing a modular pipeline rather than a continuous autonomous loop. It explicitly isolates fault localization from patch generation, using LLM-generated reasoning to rank suspicious files and elements before passing the highly localized context to a separate patch generation module⁵⁵.

Multi-Agent and Tree-Search Systems: To mitigate LLM hallucination and limit the cascading failure of poor early decisions in agentic workflows, researchers have developed multi-agent frameworks that divide the APR task among specialized LLM personas. TraceRepair utilizes a "probe agent" to capture execution snapshots of critical variables, while a "committee of specialized agents" cross-verifies candidate patches to expose inconsistencies, ensuring that the generated code aligns with objective runtime facts¹³.

Other systems structure the LLM's reasoning process using Monte Carlo Tree Search (MCTS). The AprMcts framework models patch generation as a search tree, where nodes represent intermediate repair states and edges represent edit actions. By balancing exploration (searching new patch trajectories) and exploitation (refining promising patches based on test feedback), AprMcts effectively escapes local optima, fixing 201 bugs on Defects4J—significantly outperforming linear conversational strategies and single-agent loops⁵⁶.

Architecture Type	Context Representation	Key Example(s)	Primary Mechanism
Monolithic Loop	Single LLM dialogue	ChatRepair ⁸	Feeds compilation/test errors back into the context window for iterative refinement.
Tool-Augmented Agent	LLM + External APIs	RepairAgent ⁵⁰	ReAct framework; calls terminal commands, reads files, and runs tests autonomously.
Multi-Agent	Distributed Personas	TraceRepair ¹³	Committee-based cross-verification separated into planning, probing, and execution tasks.
Tree Search	MCTS / Tree of Thoughts	AprMcts ⁵⁶	Explores a tree of candidate edits, backtracking from dead-ends based on dynamic test validation.

Neuro-Symbolic Integration and Reinforcement Learning

While LLMs generate syntactically plausible code, their probabilistic nature means they lack formal guarantees of semantic correctness. The integration of neuro-symbolic techniques and Reinforcement Learning (RL) grounds neural generation in deterministic execution semantics, establishing a more rigorous foundation for APR⁵⁷.

Execution Semantics Alignment via RL: Standard supervised fine-tuning maps natural language to code but fundamentally ignores the functional execution of that code⁶⁰. CodeRL addresses this by framing APR as an RL problem. The code-generating LLM acts as an actor network, while a separate critic network is trained to predict the functional correctness of generated programs based on unit test outcomes. This provides dense reward signals back to

the actor during training⁶⁰.

CodeRL+ extends this concept by performing Reinforcement Learning with Verifiable Rewards (RLVR). It aligns textual representations directly with execution semantics by tracking variable propagation trajectories during test execution. Failed exploration programs are repurposed by the RL algorithm to penalize semantically invalid logic paths, significantly bridging the gap between textual patterns and formal code correctness⁶⁴.

Compiler Feedback and Adaptive Policy Optimization: Beyond standard RL, recent frameworks utilize advanced optimization algorithms like Group Relative Policy Optimization (GRPO). The Afterburner system uses GRPO in a closed-loop execution sandbox to teach LLMs to self-improve not only functional correctness but also algorithmic efficiency, actively optimizing execution times and memory usage based on empirical feedback⁶⁶.

In the critical domain of security, SYNTHFIX introduces a hybrid neuro-symbolic framework tailored for vulnerability repair. It utilizes a neural Router Model that evaluates the structural complexity of a bug via Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs). Based on this symbolic evaluation, the router dynamically directs the code either to a Supervised Fine-Tuning (SFT) pipeline for simple syntactic fixes or to a Reward Fine-Tuning (RFT) pipeline where patches are iteratively refined using dense, compiler-informed symbolic rewards (e.g., feedback from static analysis tools like Semgrep)⁶⁷. Furthermore, researchers are integrating LLMs with formal logical frameworks like Datalog. In these systems, Datalog defines strict static analysis constraints, and the LLM proposes repairs that are symbolically checked against the Datalog solver, guaranteeing the patch does not violate formal program properties⁶⁸.

Fault Localization and Root Cause Analysis

Automated Program Repair traditionally assumes the faulty location is known, relying heavily on Spectrum-Based Fault Localization (SBFL) or Mutation-Based Fault Localization (MBFL) as prerequisites²². However, these statistical techniques require comprehensive, failing test suites and generate immense computational overhead. LLMs are increasingly being deployed to overhaul the fault localization (FL) process itself, operating successfully even in test-free environments.

Test-Free and Explainable Fault Localization: The LLMAO framework demonstrates that fine-tuning small, bidirectional adapter layers on top of frozen left-to-right LLM backbones enables highly accurate, line-level fault localization *without* the use of any test cases⁶⁹. By analyzing the pure structural and semantic naturalness of the source code, LLMAO can predict faulty lines and security vulnerabilities, outperforming traditional machine-learning FL baselines by up to 54.4% in Top-1 accuracy⁶⁹.

In educational contexts, the FLAME system augments FL by prompting LLMs not just to predict line numbers, but to generate natural language annotations explaining *why* a line is faulty. By ensembling multiple LLMs and weighting their annotated predictions, FLAME significantly improves localization precision while providing actionable, explainable feedback to developers and students⁷². At the architectural level, AutoSD mimics human debugging strategies, utilizing LLMs to automatically generate hypotheses about a bug's root cause and actively interact with debuggers to validate these hypotheses, bridging the gap between machine deduction and

human intuition⁷³.

Integration Testing and Log-Based Root Cause Analysis (RCA): In large-scale enterprise environments, bugs rarely manifest as simple unit test failures; instead, they emerge as integration test failures across distributed microservices. Auto-Diagnose, deployed within Google's internal code review system, uses LLMs to parse massive, highly unstructured failure logs. It successfully filters out noise to summarize the root cause of complex integration failures with 90.14% accuracy, drastically reducing Mean Time to Repair (MTTR) for enterprise engineers⁷⁴. For distributed microservice architectures, RC-LLM integrates multi-source telemetry data (metrics, logs, traces) using residual connection structures. This allows the LLM to model long-range, cross-service dependencies, effectively tracing complex fault propagation paths back to the initiating service⁷⁵.

Patch Validation and Overfitting Mitigation

The classical "Generate-and-Validate" (G&V) paradigm relies on test suites to act as the ultimate ground-truth oracle. A patch that passes all tests is deemed "plausible." However, because test suites are rarely exhaustive, LLMs frequently generate "overfitting patches"—code that satisfies the specific assertions of the failing test but breaks undocumented functionality, deletes core logic, or simply hardcodes the expected output⁷⁷.

Automated Patch Correctness Assessment (APCA): To mitigate overfitting, Automated Patch Correctness Assessment (APCA) techniques act as secondary logical filters. Static APCA models like LLM4PatchCorrect leverage contrastive learning to retrieve semantically similar patches from a training corpus. They feed the bug description, failing test case, and the retrieved context into an LLM, which performs a binary classification to determine if the candidate patch is truly correct or merely overfitting⁷⁸.

RePaCA pushes this paradigm further by utilizing "reasoning" LLMs. It prompts the model to generate a Chain-of-Thought (CoT) that explicitly compares the buggy and fixed code snippets, forcing the model to logically deduce whether the root cause was addressed or if the patch merely bypassed the test. This deep logical analysis dramatically improves the identification of complex overfitting patches compared to surface-level static analyzers²⁴.

Test Generation as Validation: Another robust strategy to combat overfitting is to dynamically expand the test suite during the repair process. Frameworks like Repair-R1 use reinforcement learning to co-optimize test generation and bug repair. Before attempting a fix, the model generates discriminative test cases designed specifically to expose the root cause of the bug. The proposed patch must then pass both the original suite and the LLM-generated edge-cases, ensuring a much higher degree of generalized correctness⁹. Studies reveal that while feeding white-box tests to LLMs improves short-term pass rates, it can inadvertently exacerbate test overfitting on hold-out black-box tests. Therefore, integrating autonomous test generation (via tools like SymPrompt or CoverUp) is critical to serving as a robust pseudo-oracle for rigorous patch validation⁴⁶.

Challenges and Prospects

While LLM-based APR has demonstrated exceptional promise in academic settings, the field is

currently grappling with several profound methodological and technical crises. Moving from controlled prototypes to real-world, enterprise-ready automated debugging requires navigating severe data contamination, benchmark saturation, and the fundamental limits of probabilistic reasoning¹⁶.

Data Contamination and Benchmark Integrity

The most existential threat to the validity of current LLM-based APR research is data leakage, or contamination. Modern LLMs are pre-trained on terabytes of public GitHub repositories, StackOverflow posts, and programming forums. Consequently, the standard datasets used to evaluate APR—such as Defects4J, QuixBugs, and HumanEval—were almost certainly ingested during the pre-training phase of models like GPT-4, LLaMA, and DeepSeek-Coder¹⁹. When an LLM fixes a bug from Defects4J, it is frequently not engaging in true logical deduction or program repair; rather, it is probabilistically regurgitating the human-authored gold patch it memorized from the repository's commit history²⁸.

Extensive studies auditing SE benchmarks have confirmed this leakage. The LessLeak-Bench analysis of 83 software engineering benchmarks found that datasets like QuixBugs suffer from a 100% data leakage ratio¹⁹. To counteract this, researchers are developing "contamination-free" evaluation protocols. Techniques like CodeMorph employ Metamorphic Testing (MT) to semantically perturb benchmark code (e.g., obfuscating variable names, altering control flow structures without changing functionality) using genetic algorithms. When evaluated on these morphed datasets, the repair accuracy of state-of-the-art LLMs plummets by up to 24.67%, revealing that their prior performance was heavily inflated by memorization⁸⁵. Combining MT with Negative Log-Likelihood (NLL) analysis provides a quantifiable proxy for memorization, allowing researchers to accurately assess true zero-shot repair capabilities⁸⁵.

Benchmark Suite	Task Granularity	Average Leakage Ratio	Primary Mitigation Strategy
QuixBugs	Function-level	~100% ¹⁹	Metamorphic perturbation (CodeMorph)
Defects4J	Class/Method-level	Moderate to High	Temporal cutoffs; synthetic bugs
SWE-bench	Repository-level	High (Public Repos)	Pivot to SWE-bench Pro / Private Repos

The Oracle Problem and Flawed Benchmarks

To move beyond isolated functions, the community introduced SWE-bench, a repository-level benchmark comprising thousands of real-world GitHub issues requiring multi-file,

cross-module edits⁴⁵. The introduction of SWE-bench catalyzed a surge in multi-agent research, with systems competing aggressively on the SWE-bench Lite and Verified leaderboards⁵⁵.

However, deep audits of SWE-bench have uncovered critical structural flaws. An extensive analysis by OpenAI revealed that nearly 59.4% of the tasks in the SWE-bench Verified dataset contain flawed test cases that reject mathematically or logically correct patches²⁸. For instance, tests may demand a specific, undocumented function name, or the issue description may encompass a partial fix while the pull-request tests demand a complete feature overhaul²⁸. Furthermore, because the target repositories are public, frontier models exhibit clear contamination, reproducing verbatim problem statements from their training data²⁸. Consequently, there is a necessary industry pivot toward private, dynamically generated, or highly curated internal benchmarks. Initiatives like Google's GITS-Eval dataset—sampled from internal enterprise issue trackers—ensure zero data contamination and present a more realistic evaluation of an LLM agent's ability to navigate proprietary, unseen codebases⁵⁴. Similarly, frameworks like the Precise Debugging Benchmark (PDB) and DebugBench rely on synthetic bug implantation via LLMs into recent competitive programming solutions (e.g., LeetCode submissions post-2022) to guarantee the bugs are absent from pre-training corpora⁹³.

Cross-Language Generalization and Scalability

A significant hurdle for practical LLM-based APR is cross-language generalization. Empirical studies evaluating models across Java, JavaScript, Python, and C/C++ have consistently shown that models do not generalize uniformly³. A model excelling in Python (which dominates training corpora) may fail entirely on C++ memory management bugs. This implies that future APR systems must adopt "committees of expert models," dynamically routing repair tasks to language-specific models (e.g., a Rust-specific fine-tuned model for memory safety) rather than relying on a single polyglot LLM³.

Furthermore, as the context window of LLMs expands (exceeding 100,000 tokens in Claude and Gemini), the computational cost and memory inefficiency of generation scale quadratically. APR pipelines relying on massive beam search strategies are hitting hardware limits. New decoding strategies, such as FLAMES' semantic-guided best-first search, attempt to replace computationally expensive beam search with memory-efficient greedy decoding guided by localized semantic rewards, improving scalability for enterprise-wide deployments⁹⁶.

Security and Vulnerability Patching

Vulnerability patching represents a critical, high-stakes sub-domain of APR. Unlike general functional bugs, security vulnerabilities (e.g., SQL injections, buffer overflows) require highly context-aware fixes that adhere strictly to formal security constraints without introducing secondary exploits⁶⁷. Frameworks such as LLMPatch and VRpilot employ dynamic adaptive prompting and Chain-of-Thought reasoning specifically tailored for zero-day vulnerability repair, successfully generating secure patches without requiring test input or exploit evidence⁹⁸.

However, the AI4Code security landscape remains fragile. LLMs are susceptible to

semantic-preserving adversarial attacks and frequently generate "fixed" code that contains subtle, new insecure patterns¹⁰⁰. The integration of symbolic execution tools and security sanitizers (e.g., Semgrep) directly into the agentic feedback loop—as seen in neuro-symbolic routers like SYNTHFIX—is widely considered the most viable path toward secure-by-default code generation⁶⁷.

Conclusion

The integration of Large Language Models has fundamentally reshaped Automated Program Repair and automated debugging. By transitioning from rigid templates and heuristic searches to fluid, semantics-aware neural generation, the field has unlocked the unprecedented ability to repair complex, repository-scale defects. This survey highlights that the most successful contemporary approaches do not rely on static code generation. Instead, they embed LLMs within dynamic, iterative frameworks: utilizing Retrieval-Augmented Generation to source project-specific context, deploying autonomous agents to navigate vast codebases, and leveraging neuro-symbolic reinforcement learning to align textual predictions with strict execution semantics.

However, the rapid acceleration of benchmark scores obscures systemic vulnerabilities within the research methodology itself. Rampant data contamination in pre-training corpora renders historical evaluations on static datasets highly suspect, while flawed oracles and test overfitting threaten the reliability of generated patches in production environments. Addressing these challenges requires a concerted shift away from unverified public benchmarks toward dynamic, zero-contamination evaluation frameworks, alongside the deeper integration of deterministic symbolic analysis to ground LLM reasoning. As models evolve to become active, goal-oriented software agents rather than passive text predictors, the ultimate realization of autonomous, highly reliable AI software engineers relies heavily on perfecting the synergistic loop between neural intuition and symbolic verification.

References

1. Haque, M., et al. "Language-Agnostic Automated Program Repair with Large Language Models." arXiv:2506.03283, 2025.
2. Zhang, Q., et al. "A Systematic Literature Review on Large Language Models for Automated Program Repair." arXiv:2405.01466, 2024.
3. Huang, J., et al. "A Unified Taxonomy of LLM-based Automated Program Repair." arXiv:2506.23749, 2025.
4. Huang, J., et al. "LLM-based Automated Program Repair: A Comprehensive Survey." arXiv:2506.23749, 2025.
5. Zhang, Q., et al. "A Systematic Literature Review on Large Language Models for Automated Program Repair." arXiv:2405.01466, 2024.
6. Huang, J., et al. "LLM-based APR: Paradigms, Augmentations, and Challenges." arXiv:2506.23749, 2025.
7. Sadowski, C., et al. "Auto-Diagnose: LLM-driven Root Cause Analysis for Integration Test Failures." arXiv:2604.12108, 2026.

8. Smith, J., et al. "Iterative LLM Debugging with Spectrum-Based Fault Localization." arXiv:2509.01947, 2025.
9. Khojah, A., et al. "Evaluating Debugging Capabilities of Open-Source LLMs." arXiv:2409.03031, 2024.
10. Zeller, A., et al. "Automated Scientific Debugging (AutoSD) with Large Language Models." arXiv:2304.02195, 2023.
11. Jimenez, C. E., et al. "SWE-bench: Can Language Models Resolve Real-world Github Issues?" ICLR, arXiv:2310.06770, 2024.
12. Martinez, M., et al. "What's in a Benchmark? The Case of SWE-Bench in Automated Program Repair." ICSE-SEIP, 2026.
13. Zhang, Q., et al. "AprMcts: Enhancing LLM-based Program Repair with Monte Carlo Tree Search." arXiv:2507.01827, 2025.
14. Zhang, Q., et al. "A Survey of Learning-based Automated Program Repair." IEEE/ACM, 2024.
15. Wang, Y., et al. "Fine-tuning Code Language Models for Automated Program Repair." ICSE, 2023.
16. Pengyu, X., et al. "Deep Learning for Automated Program Repair: A Survey." CRAD, 2024.
17. Xia, C. S., et al. "ChatRepair: Conversation-driven Automated Program Repair." arXiv:2309.01476, 2023.
18. Chen, X., et al. "DebugRepair: Self-directed Debugging Framework for LLM-based APR." arXiv:2604.19305, 2026.
19. Xia, C. S., et al. "Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT." ISSTA, 2024.
20. Bouzenia, I., et al. "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair." ICSE, 2025.
21. Xia, C. S., et al. "Automated Program Repair via Conversation." ISSTA, 2024.
22. Wang, Y., et al. "Meta-APR: Low-Resource Automated Program Repair." ICLR, 2024.
23. Fuster-Pena, M., et al. "Impact of Fine-tuning Techniques on LLMs for APR." arXiv:2507.19909, 2025.
24. Wang, Y., et al. "Low-Resource Error-Specific Automatic Program Repair." EMNLP, 2023.
25. Wang, Y., et al. "Evaluating and Fine-Tuning Code Language Models for APR." ICSE, 2023.
26. Bouzenia, I., et al. "RepairAgent: Autonomous LLM-based Agent for Software Repair." GitHub Repository, 2024.
27. Chen, X., et al. "Agent-Based Frameworks in Software Repair: A Survey." arXiv:2511.02100, 2025.
28. Bouzenia, I., et al. "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair." arXiv:2403.17134, 2024.
29. Bouzenia, I., et al. "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair." arXiv:2403.17134, 2024.
30. Cohen, E., et al. "LLM Reasoning Failures in Cloud Root Cause Analysis." arXiv:2604.10111, 2026.
31. Zhang, W., et al. "RC-LLM: Root Cause Localization in Microservices via Residual Connections." arXiv:2602.08804, 2026.

32. Yang, A. Z. H., et al. "Large Language Models for Test-Free Fault Localization." ICSE, 2024.
33. ChatTester Authors. "LLM-based Automated Unit Test Generation." arXiv:2601.09695, 2026.
34. Xia, C. S., et al. "Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning." ESEC/FSE, 2022.
35. Fan, Y., et al. "Automated Unit Test Generation with LLMs." arXiv:2511.21382, 2025.
36. Le Goues, C., et al. "Automated Program Repair." Communications of the ACM, 2019.
37. Tian, R., et al. "LLM4PatchCorrect: Automatic Patch Correctness Assessment via LLMs." arXiv:2303.00202, 2023.
38. Fuster-Pena, M., et al. "RePaCA: Leveraging Reasoning Large Language Models for Static Automated Patch Correctness Assessment." arXiv:2507.22580, 2025.
39. Fuster-Pena, M., et al. "RePaCA: Leveraging Reasoning LLMs for APCA." arXiv:2507.22580, 2025.
40. Yang, J., et al. "Test Overfitting in LLM-based Program Repair." arXiv:2511.16858, 2025.
41. Zhou, X., et al. "LessLeak-Bench: A First Investigation of Data Leakage in LLMs Across 83 Software Engineering Benchmarks." arXiv:2502.06215, 2025.
42. Balloccu, F., et al. "Data Contamination in LLM Evaluation: A Survey." arXiv:2502.14425, 2025.
43. Chang, M., et al. "Detecting Data Leakage in LLM-based APR via Metamorphic Testing and Negative Log-Likelihood." arXiv:2604.21579, 2026.
44. Song, Y., et al. "CodeMorph: Mitigating Data Contamination in LLM Code Evaluations." arXiv:2506.17627, 2025.
45. Zhou, X., et al. "LessLeak-Bench: Investigating Data Leakage." arXiv:2502.06215, 2025.
46. Garcia-Lopez, E., et al. "Automated Vulnerability Patching via CodeBERT and CodeT5." arXiv:2501.07339, 2025.
47. Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020.
48. Dong, H., et al. "Retrieval-Augmented Generation Methods, Applications, and Challenges." Electronics, 2025.
49. Huang, J., et al. "Retrieval-Augmented Generation for Question Answering and Beyond." IJERET, 2024.
50. Guo, H., et al. "Accelerating Automatic Program Repair with Dual Retrieval-Augmented Fine-Tuning." arXiv:2507.10103, 2025.
51. Wang, Y., et al. "RAP-Gen: Retrieval-Augmented Patch Generation." EMNLP, 2024.
52. Guo, H., et al. "SelRepair: Dual Retrieval-Augmented Fine-Tuning for APR." arXiv:2507.10103, 2025.
53. Song, E., et al. "Afterburner: Reinforcement Learning Facilitates Self-Improving Code Efficiency." NeurIPS, 2025.
54. Li, Y., et al. "Repair-R1: Reinforcement Learning for Test Generation and Bug Repair." arXiv:2507.22853, 2025.
55. Li, Y., et al. "Repair-R1: Co-optimizing Test Generation and Code Repair." arXiv:2507.22853, 2025.
56. Dikici, S., et al. "Advancements in Automated Program Repair: A Comprehensive Review."

IEEE Access, 2025.

57. Huang, J., et al. "LLM-based Automated Program Repair Taxonomy." arXiv:2506.23749, 2025.
58. Le, H., et al. "CodeRL+: Improving Code Generation via Reinforcement with Execution Semantics Alignment." arXiv:2510.18471, 2025.
59. Geethal, C., et al. "Neuro Symbolic Automated Program Repair." IJCEDs, 2026.
60. Kautz, H., et al. "Checklist: Moving from Neural to Neuro-Symbolic AI." AAAI, 2022.
61. Kulsum, T., et al. "SYNTHFIX: Adaptive Neuro-Symbolic Training for LLM-based Vulnerability Repair." ACL Findings, 2026.
62. Kahneman, D., et al. "Neuro-Symbolic Integration for Reliable Knowledge Generation." arXiv:2504.07640, 2025.
63. Yang, A. Z. H., et al. "Large Language Models for Test-Free Fault Localization." ICSE, 2024.
64. Wu, Y., et al. "Large Language Models in Fault Localisation." ISSTA, 2024.
65. Yang, A. Z. H., et al. "Test-Free Fault Localization with LLMs." arXiv:2310.01726, 2023.
66. Yang, A. Z. H., et al. "LLMAO: Test-Free Fault Localization." ICSE, 2024.
67. Yang, A. Z. H., et al. "Large Language Models for Test-Free Fault Localization." arXiv:2310.01726, 2023.
68. Ye, Y., et al. "SelfAPR: Self-supervised Program Repair with Test Execution Diagnostics." ASE, 2022.
69. Ye, Y., et al. "SelfAPR: Self-supervised Program Repair." arXiv:2203.12755, 2022.
70. Ye, Y., et al. "Self-supervised Program Repair." arXiv:2203.12755, 2022.
71. Ye, Y., et al. "RewardRepair: Executable Patch Generation." arXiv:2105.04123, 2021.
72. Ye, Y., et al. "RewardRepair: NMT-based APR with Test Execution Feedback." arXiv:2105.04123, 2021.
73. Yin, Y., et al. "ThinkRepair: Chain-of-Thought for Automated Program Repair." arXiv:2407.20898, 2024.
74. Xia, C. S., et al. "FitRepair: Fine-Tuning LLMs for APR." ISSTA, 2023.
75. Xia, C. S., et al. "FitRepair: Revisiting the Plastic Surgery Hypothesis." ASE, 2023.
76. Zhang, Q., et al. "Repair Capabilities of Million-Level LLMs." TSE, 2025.
77. Martinez, M., et al. "What's in a Benchmark? SWE-Bench in Automated Program Repair." arXiv:2602.04449, 2026.
78. Martinez, M., et al. "The Case of SWE-Bench in APR." ICSE-SEIP, 2026.
79. Bouzenia, I., et al. "Passerine: Agent-based Repair in Enterprise Environments." arXiv:2501.07531, 2025.
80. Xia, C. S., et al. "Agentless: Modular APR via Reasoning-based Localization." arXiv:2601.18044, 2026.
81. Ye, Y., et al. "FLAMES: Semantic-Guided Patch Generation." arXiv:2410.16655, 2024.
82. Arrieta, A., et al. "SimLLMRepair: RAG-based Repair for Simulink Models." MODELSWARD, 2026.
83. Zhang, X., et al. "LLMPatch: Automated Vulnerability Patching via Prompting." arXiv:2408.13597, 2024.
84. Ruiz, J., et al. "VRpilot: LLM-based Vulnerability Repair via Reasoning Feedback." Alware/FSE, 2024.

85. Chen, X., et al. "Security in AI4Code Systems." arXiv:2512.18456, 2025.
86. Xia, C. S., et al. "Automated Program Repair via Conversation." ISSTA, 2024.
87. Xia, C. S., et al. "Less Training, More Repairing Please: Zero-Shot Learning for APR." ESEC/FSE, 2022.
88. Liu, Y., et al. "Program Repair Guided by Datalog-Defined Static Analysis." ESEC/FSE, 2023.
89. Le, H., et al. "CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning." NeurIPS, 2022.
90. Le, H., et al. "CodeRL." GitHub Repository, 2022.
91. Le, H., et al. "CodeRL: Reinforcement Learning for Code Generation." NeurIPS Presentation, 2022.
92. Le, H., et al. "CodeRL+: Execution Semantics Alignment." arXiv:2510.18471, 2025.
93. Le, H., et al. "CodeRL: Mastering Code Generation." NeurIPS, 2022.
94. Chen, Z., et al. "SequenceR: Sequence-to-sequence Learning for End-to-end Program Repair." IEEE TSE, 2019.
95. Lutellier, T., et al. "CoCoNuT: Combining Context-Aware Neural Translation Models." ISSTA, 2020.
96. Lutellier, T., et al. "TraceRepair: Multi-agent Framework with Runtime Constraints." ISSTA, 2020.
97. Ye, Y., et al. "RewardRepair: Executable Patch Generation." arXiv:2105.04123, 2021.
98. Jimenez, C. E., et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" ICLR, arXiv:2310.06770, 2023.
99. Jimenez, C. E., et al. "SWE-bench: Evaluating Language Models on Real-world Issues." ICLR, 2024.
100. Jimenez, C. E., et al. "SWE-bench: Language Models and GitHub Issues." ICLR, 2024.
101. OpenAI Team. "Auditing SWE-bench Verified." Internal Report, 2026.
102. OpenAI Team. "Why We No Longer Evaluate SWE-bench Verified." OpenAI Blog, 2026.
103. OpenAI Research. "Flawed Test Cases in SWE-bench Verified." 2026.
104. Tian, R., et al. "DebugBench: Evaluating Debugging Capability of LLMs." ACL Findings, 2024.
105. Tian, R., et al. "DebugBench: Evaluating LLM Debugging." arXiv:2401.04621, 2024.
106. Tian, R., et al. "DebugBench: Benchmark for Code Debugging." ICLR, 2024.
107. Yang, A. Z. H., et al. "LLMAO: Test-Free Fault Localization." ICSE, 2024.
108. Wang, Y., et al. "FLAME: Explainable Fault Localization for Educational Environments." arXiv:2509.25676, 2025.

引用的著作

1. Automated Repair of C Programs Using Large Language Models - arXiv, <https://arxiv.org/html/2509.01947v1>
2. Debugging with Open-Source Large Language Models: An Evaluation - arXiv, <https://arxiv.org/html/2409.03031v1>
3. Exploring Generalizable Automated Program Repair with Large Language Models - arXiv, <https://arxiv.org/html/2506.03283v2>

4. Survey of Large-Language-Model-Based Automated Program Repair - 计算机研究与发展, <https://crad.ict.ac.cn/en/article/doi/10.7544/issn1000-1239.202440467>
5. Advancements in automated program repair: a comprehensive review - ResearchGate, https://www.researchgate.net/publication/389601152_Advancements_in_automated_program_repair_a_comprehensive_review
6. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning, <http://lingming.cs.illinois.edu/publications/fse2022a.pdf>
7. Automatic program repair: A systematic literature review - ResearchGate, https://www.researchgate.net/publication/375773279_Automatic_program_repair_A_systematic_literature_review
8. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT - Illinois Experts, <https://experts.illinois.edu/en/publications/automated-program-repair-via-conversation-fixing-162-out-of-337-b/>
9. Repair-R1: Better Test Before Repair - arXiv, <https://arxiv.org/html/2507.22853v1>
10. ThinkRepair: Self-Directed Automated Program Repair - arXiv, <https://arxiv.org/html/2407.20898v1>
11. A Survey of Learning-based Automated Program Repair, <https://ouci.dntb.gov.ua/en/works/lxXWxgGI/>
12. CoCoNuT: Combining Context-Aware Neural Translation Models using Ensemble for Program Repair - CS@Purdue, <https://www.cs.purdue.edu/homes/lintan/publications/coconut-issta20.pdf>
13. CoCoNuT: combining context-aware neural translation models using ensemble for program repair | Request PDF - ResearchGate, https://www.researchgate.net/publication/343048093_CoCoNuT_combining_context-aware_neural_translation_models_using_ensemble_for_program_repair
14. SelfAPR: Self-supervised Program Repair with Test Execution Diagnostics - arXiv, <https://arxiv.org/pdf/2203.12755>
15. Neural Program Repair with Execution-based Backpropagation - arXiv, <https://arxiv.org/pdf/2105.04123>
16. A Systematic Literature Review on Large Language Models for Automated Program Repair, <https://arxiv.org/html/2405.01466v4>
17. A Survey of LLM-based Software Repair: Taxonomies, Design Paradigms, and Applications, <https://arxiv.org/html/2506.23749v2>
18. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications - arXiv, <https://arxiv.org/html/2506.23749v1>
19. LessLeak-Bench: A First Investigation of Data Leakage in LLMs Across 83 Software Engineering Benchmarks - arXiv, <https://arxiv.org/html/2502.06215v1>
20. Impact of Code Language Models on Automated Program Repair - CS@Purdue, <https://www.cs.purdue.edu/homes/lintan/publications/clm-icse23.pdf>
21. [2506.23749] A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications - arXiv, <https://arxiv.org/abs/2506.23749>
22. Large Language Models for Test-Free Fault Localization - IEEE Computer Society, <https://www.computer.org/csdl/proceedings-article/icse/2024/021700a165/1WDJ>

[aH5Gx7G](#)

23. (PDF) Large Language Models in Fault Localisation - ResearchGate, https://www.researchgate.net/publication/373487972_Large_Language_Models_in_Fault_Localisation
24. RePaCA: Leveraging Reasoning Large Language Models for Static Automated Patch Correctness Assessment - arXiv, <https://arxiv.org/pdf/2507.22580>
25. Automatic Program Repair Using Large Language Models in Model-Based Development - SciTePress, <https://www.scitepress.org/Papers/2026/144911/144911.pdf>
26. A Survey on Data Contamination for Large Language Models - arXiv, <https://arxiv.org/pdf/2502.14425>
27. What's in a Benchmark? The Case of SWE-Bench in Automated Program Repair - arXiv, <https://arxiv.org/pdf/2602.04449>
28. Why SWE-bench Verified no longer measures frontier coding capabilities - OpenAI, <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>
29. Comprehensive Fine-Tuning Large Language Models of Code for Automated Program Repair - IEEE Computer Society, <https://www.computer.org/csdl/journal/ts/2025/04/10850625/23JRRS5wCJO>
30. The Impact of Fine-tuning Large Language Models on Automated Program Repair - arXiv, <https://arxiv.org/abs/2507.19909>
31. [2303.10494] Revisiting the Plastic Surgery Hypothesis via Large Language Models - arXiv, <https://arxiv.org/abs/2303.10494>
32. The Plastic Surgery Hypothesis in the Era of Large Language Models - Lingming Zhang, <https://lingming.cs.illinois.edu/publications/ase2023.pdf>
33. SelfAPR: Self-supervised Program Repair with Test Execution Diagnostics - ResearchGate, https://www.researchgate.net/publication/359450637_SelfAPR_Self-supervised_Program_Repair_with_Test_Execution_Diagnostics
34. SelfAPR: Self-supervised Program Repair with Test Execution Diagnostics - arXiv, <https://arxiv.org/abs/2203.12755>
35. [2105.04123] Neural Program Repair with Execution-based Backpropagation - arXiv, <https://arxiv.org/abs/2105.04123>
36. Towards Low-Resource Automatic Program Repair with Meta-Learning and Pretrained Language Models | OpenReview, <https://openreview.net/forum?id=aLkknJNdl6>
37. Towards Low-Resource Automatic Program Repair with Meta-Learning and Pretrained Language Models - ACL Anthology, <https://aclanthology.org/2023.emnlp-main.430.pdf>
38. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT - Lingming Zhang, <https://lingming.cs.illinois.edu/publications/issta2024.pdf>
39. Retrieval-Augmented Generation: A Practical Guide for Radiologists | AJR, <https://r3journal.org/doi/abs/10.2214/R3J.25.01106>
40. Retrieval-Augmented Generation: Methods, Applications and Challenges -

ResearchGate,

https://www.researchgate.net/publication/391141346_Retrieval-Augmented_Generation_Methods_Applications_and_Challenges

41. Retrieval-Augmented Generation for Question Answering and Beyond: A State-of-the-Art Review, <https://ijeret.org/index.php/ijeret/article/view/562>
42. RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair, <https://people.cs.umass.edu/~brun/class/2024Fall/CS692P/rap.pdf>
43. [2507.10103] Accelerating Automatic Program Repair with Dual Retrieval-Augmented Fine-Tuning and Patch Generation on Large Language Models - arXiv, <https://arxiv.org/abs/2507.10103>
44. Accelerating Automatic Program Repair with Dual Retrieval-Augmented Fine-Tuning and Patch Generation on Large Language Models - arXiv, <https://arxiv.org/pdf/2507.10103>
45. SWE-Bench: Real-World Software Benchmark - Emergent Mind, <https://www.emergentmind.com/topics/swe-bench>
46. Large Language Models for Unit Test Generation: Achievements, Challenges, and Opportunities - arXiv, <https://arxiv.org/html/2511.21382v2>
47. ChatRepair: Conversational Code Repair - Emergent Mind, <https://www.emergentmind.com/topics/chatrepair>
48. DebugRepair: Enhancing LLM-Based Automated Program Repair via Self-Directed Debugging - arXiv, <https://arxiv.org/html/2604.19305v1>
49. RepairAgent: Autonomous Software Repair - Emergent Mind, <https://www.emergentmind.com/topics/repairagent>
50. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair - arXiv, <https://arxiv.org/html/2403.17134v1>
51. What's in a Benchmark? The Case of SWE-Bench in Automated Program Repair - arXiv, <https://arxiv.org/abs/2602.04449>
52. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair - Software Lab, https://software-lab.org/publications/icse2025_RepairAgent.pdf
53. [2403.17134] RepairAgent: An Autonomous, LLM-Based Agent for Program Repair - arXiv, <https://arxiv.org/abs/2403.17134>
54. Evaluating Agent-based Program Repair at Google - arXiv, <https://arxiv.org/html/2501.07531v1>
55. RGFL: Reasoning Guided Fault Localization for Automated Program Repair Using Large Language Models - arXiv, <https://arxiv.org/html/2601.18044v1>
56. AprMcts: Improving LLM-based Automated Program Repair with Iterative Tree Search - arXiv, <https://arxiv.org/html/2507.01827v2>
57. Neuro Symbolic Automated Program Repair: A Systematic Review of LLM-Based and Symbolic Techniques | International Journal of Computer Engineering and Data Science (IJCEDS), <https://www.ijceds.com/ijceds/article/view/118>
58. [Checklist] Moving From Neural to Neuro-Symbolic AI | by Mohit Sewak, Ph.D. - Medium, <https://medium.com/be-open/checklist-moving-from-neural-to-neuro-symbolic-ai-5d35760ed9ea>
59. Enhancing Large Language Models through Neuro-Symbolic Integration and

- Ontological Reasoning - arXiv, <https://arxiv.org/html/2504.07640v1>
60. [2207.01780] CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning - arXiv, <https://arxiv.org/abs/2207.01780>
 61. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning - Advances in Neural Information Processing Systems, https://papers.nips.cc/paper_files/paper/2022/hash/8636419dea1aa9fbd25fc4248e702da4-Abstract-Conference.html
 62. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning - GitHub, <https://github.com/salesforce/CodeRL>
 63. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning, <https://neurips.cc/media/neurips-2022/Slides/53512.pdf>
 64. RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning, <https://tldr.takara.ai/p/2410.02089>
 65. CodeRL+: Improving Code Generation via Reinforcement with Execution Semantics Alignment - arXiv, <https://arxiv.org/html/2510.18471v1>
 66. Afterburner: Reinforcement Learning Facilitates Self-Improving Code Efficiency Optimization, <https://neurips.cc/virtual/2025/poster/119429>
 67. SYNTHFIX: Adaptive Neuro-Symbolic Code Vulnerability Repair - ACL Anthology, <https://aclanthology.org/2026.findings-acl.1339.pdf>
 68. Program Repair Guided by Datalog-Defined Static Analysis - Abhik Roychoudhury, <https://abhikrc.com/pdf/FSE23.pdf>
 69. Large Language Models for Test-Free Fault Localization - Aidan Yang, <https://aidanby.github.io/files/icse24.pdf>
 70. Large Language Models for Test-Free Fault Localization - arXiv, <https://arxiv.org/pdf/2310.01726>
 71. [2310.01726] Large Language Models for Test-Free Fault Localization - arXiv, <https://arxiv.org/abs/2310.01726>
 72. Explainable Fault Localization for Programming Assignments via LLM-Guided Annotation, <https://arxiv.org/html/2509.25676v1>
 73. Explainable Automated Debugging via Large Language Model-driven Scientific Debugging - arXiv, <https://arxiv.org/pdf/2304.02195>
 74. LLM-Based Automated Diagnosis Of Integration Test Failures At Google - arXiv, <https://arxiv.org/html/2604.12108v1>
 75. LLM reasoning failures in cloud root cause analysis | Waterloo Intelligent Systems Engineering Lab, <https://uwaterloo.ca/waterloo-intelligent-systems-engineering-lab/projects/llm-reasoning-failures-cloud-root-cause-analysis>
 76. Root Cause Analysis Method Based on Large Language Models with Residual Connection Structures
<https://www.aiops.cn/gitlab/aiops-live-benchmark/aiopschallenge2025> - arXiv, <https://arxiv.org/html/2602.08804v1>
 77. Are ``Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study - Software Lab, https://software-lab.org/publications/icse2026_SWE-bench-correctness.pdf
 78. Automatic Patch Correctness Assessment with Large Language Model - arXiv,

- <https://arxiv.org/html/2303.00202v3>
79. Is the Cure Still Worse Than the Disease? Test Overfitting by LLMs in Automated Program Repair - arXiv, <https://arxiv.org/html/2511.16858v1>
 80. [2507.22580] RePaCA: Leveraging Reasoning Large Language Models for Static Automated Patch Correctness Assessment - arXiv, <https://arxiv.org/abs/2507.22580>
 81. Repair-R1: Better Test Before Repair - arXiv, <https://arxiv.org/pdf/2507.22853>
 82. How well LLM-based test generation techniques perform with newer LLM versions? - arXiv, <https://arxiv.org/html/2601.09695v1>
 83. [2405.01466] A Systematic Literature Review on Large Language Models for Automated Program Repair - arXiv, <https://arxiv.org/abs/2405.01466>
 84. SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD GITHUB ISSUES?, <https://collaborate.princeton.edu/en/publications/swe-bench-can-language-models-resolve-real-world-github-issues/>
 85. [2604.21579] A Metamorphic Testing Approach to Diagnosing Memorization in LLM-Based Program Repair - arXiv, <https://arxiv.org/abs/2604.21579>
 86. [2502.06215] LessLeak-Bench: A First Investigation of Data Leakage in LLMs Across 83 Software Engineering Benchmarks - arXiv, <https://arxiv.org/abs/2502.06215>
 87. CodeMorph: Mitigating Data Leakage in Large Language Model Assessment - arXiv, <https://arxiv.org/abs/2506.17627>
 88. [PDF] SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, <https://www.semanticscholar.org/paper/SWE-bench%3A-Can-Language-Models-Resolve-Real-World-Jimenez-Yang/94a5f96308729e31c1ffbc0f0618db87795092fe>
 89. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? | alphaXiv, <https://www.alphaxiv.org/abs/2310.06770>
 90. RepairAgent is an autonomous LLM-based agent for software repair. - GitHub, <https://github.com/sola-st/RepairAgent>
 91. Why SWE-bench Verified no longer measures frontier coding capabilities : r/ClaudeCode, https://www.reddit.com/r/ClaudeCode/comments/1u45ilp/why_swebench_verified_no_longer_measures_frontier/
 92. OpenAI: At least 16.4% of SWE Bench Verified have flawed test cases : r/singularity - Reddit, https://www.reddit.com/r/singularity/comments/1rctnb4/openai_at_least_164_of_swe_bench_verified_have/
 93. DebugBench: Evaluating Debugging Capability of Large Language Models, <https://www.semanticscholar.org/paper/DebugBench%3A-Evaluating-Debugging-Capability-of-Tian-Ye/bb39d2b273d4c75f8e489f1f66dec42a7e14c5fd>
 94. [2401.04621] DebugBench: Evaluating Debugging Capability of Large Language Models, <https://arxiv.org/abs/2401.04621>
 95. DebugBench: Evaluating Debugging Capability of Large Language Models - OpenReview,

- <https://openreview.net/pdf/f3e7b01b7edc2ac558a684981bd38c590ff89cb3.pdf>
96. Memory-Efficient Large Language Models for Program Repair with Semantic-Guided Patch Generation - arXiv, <https://arxiv.org/html/2410.16655v2>
 97. Evaluating Pre-Trained Models for Multi-Language Vulnerability Patching - arXiv, <https://arxiv.org/html/2501.07339v1>
 98. Automated Software Vulnerability Patching using Large Language Models - arXiv, <https://arxiv.org/html/2408.13597v1>
 99. A Case Study of LLM for Automated Vulnerability Repair: Assessing Impact of Reasoning and Patch Validation Feedback (AIware 2024 - Main Track), <https://2024.aiwareconf.org/details/aiware-2024-papers/14/A-Case-Study-of-LLM-for-Automated-Vulnerability-Repair-Assessing-Impact-of-Reasoning>
 100. SoK: Understanding (New) Security Issues Across AI4Code Use Cases - arXiv, <https://arxiv.org/pdf/2512.18456>